

Homework Assignment 1

Sensorimotor Neurotechnology, March 2025

Using a Kalman filter to decode brain signals

Homework Assignment 1 is meant to give you some hands-on experience using a Kalman filter for neural decoding. I will first give a brief review on the Kalman filter and its application to body control. I will then describe how it can be modified to become a decoder for brain control. Finally, I include the actual assignment: You will apply the equations described in the previous two sections to decode kinematics from real brain signals (neurons in Somatosensory Area 2). The questions are also designed to give you some intuition for how the decoder works. Enjoy!

Kalman filter and body control

Let's briefly review how a Kalman filter can perform state estimation in the context of body control. We will do so in discrete time. To keep with the notation I used in lecture, I will write $t + 1$ to refer to the *current* time step and t to refer to the *previous* time step (You may also see this written as t and $t - 1$, respectively; note that this difference in notation means the same thing). In our hypothetical example, an actor wishes to track how the state of their arm $\mathbf{x}_t$ changes given a motor command $\mathbf{u}_t$. There are three main equations that formalize this process: the state update equation; the sensory observation equation; and the state estimation equation. For a full overview, I suggest reviewing the lecture slides.

State update equation. This equation formalizes how the state of the arm changes from time point t to time point $t + 1$.

$$\mathbf{x}_{t+1} = A\mathbf{x}_t + B\mathbf{u}_t \quad (1)$$

Where the current state of the limb, $\mathbf{x}_{t+1} = \{x_{t+1}, y_{t+1}, \dot{x}_{t+1}, \dot{y}_{t+1}\}$, is dependent upon the previous limb state $\mathbf{x}_t$ and the motor command $\mathbf{u}_t$. A is the *state transition matrix*, which governs how the state changes given the current positions and velocities. B is the *motor transition matrix*, which governs how the motor command (often formalized as an acceleration) modifies the positions and velocities. See the lecture for more details on both matrices. Note that this equation refers to the *actual* state of the arm, which is not directly observable by the brain.

Sensory observation equation. When the arm moves, this change is registered by the sense organs (e.g., muscle spindles, retina, etc.) and communicated to the brain via peripheral nerves (i.e., sensory feedback). This is called a sensory observation, and can be formalized as follows.

$$\mathbf{y}_{t+1} = H\mathbf{x}_{t+1} + \mathbf{q} \quad (2)$$

H is the *observation matrix*, which maps the current limb state into sensory feedback $\mathbf{y}_{t+1}$. This signal is not perfect and is known to be corrupted by additive observation noise $\mathbf{q}$, which is assumed to be Gaussian, $\mathbf{q} \sim N(0, Q)$; Q is the covariance matrix of the noise for each observation dimension. Unlike the actual state of the limb, the sensory feedback is observable by the brain.

The forward model. As discussed in lecture, while the nervous system has access to sensory feedback, the actual current state of the limb is inaccessible. Furthermore, sensory feedback

delays (~30-50 ms for proprioception) are known to destabilize body control. The Kalman filter therefore assumes that the user has a forward model, which makes an *a priori* estimate of the *current* state of the limb x_{t+1}^* from the motor command and internal models of limb dynamics, $\hat{A}$ and $\hat{B}$. We can therefore reformulate the state update equation as follows:

$$x_{t+1}^* = \hat{A}x_t^* + \hat{B}u_t + w \quad (3)$$

Where x_{t+1}^* is the *a priori* estimate about the state of the limb from the forward model and w is the prediction noise that is assumed to be Gaussian, $w \sim N(0, W)$. The forward model also estimates the expected sensory feedback.

$$y_{t+1}^* = \hat{H}x_{t+1}^* \quad (4)$$

Where $\hat{H}$ is the internal estimate of the observation matrix that maps the *a priori* estimate of the limb state x_{t+1}^* to the expected sensory observation y_{t+1}^* .

State estimation equation. This brings us to perhaps the most important part of the Kalman filter (at least for decoding): state estimation. Up to now, the nervous system has created *a priori* estimates using a forward model and received sensory observations from the periphery. The question is, how do we use both pieces of information to derive an optimal *a posteriori* estimate of the limb state? The Kalman filter formalizes this process as follows:

$$\hat{x}_{t+1} = x_{t+1}^* + K_{t+1}(y_{t+1} - y_{t+1}^*) \quad (5)$$

Where the *a posteriori* state estimate $\hat{x}_{t+1}$ is derived from the *a priori* estimate x_{t+1}^* and the weighted prediction error between the actual and estimated sensory observation. The parameter that weighs this prediction error (i.e., discrepancy between actual and predicted observation) is appropriately called the *Kalman gain*.

So, how do we set the proper value of the Kalman gain? To fully understand this, it is crucial for us to go into one aspect of the estimation process that I glossed over in lecture: all estimates are *probabilistic*. That is, they are not single point estimates but contain uncertainty about the state of the body. From a Bayesian perspective, the optimal thing to do would be to weigh each source of body state information (i.e., prediction and observation) by how reliable they are (i.e., the inverse of their uncertainty). In lecture, I (crudely) stated that K is proportional to the ratio between prediction uncertainty and total uncertainty. Let's go into more detail on how we *actually* formalize this intuition mathematically. At a given time, the Kalman gain K_{t+1} is:

$$K_{t+1} = C_{t+1}H^T(HC_{t+1}H^T + Q)^{-1} \quad (6)$$

Where the magnitude of the Kalman gain K_{t+1} is dependent upon the relative proportion of uncertainty in the sensory observation, Q , and the uncertainty of the *a priori* estimate, C_{t+1} . You'll note here that the uncertainty of the sensory observation remains static throughout the movement, but the uncertainty of the *a priori* estimate evolves over time. When the Kalman gain is high, prediction errors will have more influence on the *a posteriori* estimate. As the agent becomes more certain in their predictions, the Kalman gain will decrease and the influence of prediction errors on the state estimation will decrease as well.

At each time step, the covariance of the *a priori* estimate, C_{t+1} , is as follows:

$$C_{t+1} = AP_tA^T + W \quad (7)$$

Where W is the covariance of the prediction uncertainty and P_t is the covariance of the *a posteriori* estimate on the *previous* time step $\hat{x}_t$ (see below). This exemplifies the fact that the Kalman filter is a recursive algorithm, where the posterior of the previous time step t is used to derive the statistical properties of the prior at the current time step $t + 1$. Thus, the above processes depend upon the uncertainty in the *a posteriori* estimate of body state, defined as:

$$P_{t+1} = (I - K_{t+1}H)C_{t+1} \quad (8)$$

Where P_{t+1} is the current covariance matrix of the *a posteriori* state estimate. It is linked to the current *a priori* uncertainty C_{t+1} and the current value of the Kalman gain. To link Equation 8 with Equation 7, P_{t+1} then becomes P_t to derive the *a priori* covariance on the next time step. Let's now see how this algorithm can be used for neural decoding.

Kalman filter: A neural decoder for brain control

For our purposes and that of neural prosthetic control, we want to modify the above equations such that they can be used to decode movement intentions from a biological signal. How is this done? Below we will discuss a common Kalman-based decoding algorithm, which I have adapted from the paper Wu et al., 2003 presented at NeurIPS. The notation I use is a bit different from the paper, to line it up better with the lecture slides. But the algorithm is the same.

Let's say that we want to decode the intended two-dimensional kinematics of a cursor, $x_t = \{1, x_t, y_t, \dot{x}_t, \dot{y}_t\}$, from the current firing rates of 50 M1 neurons, $r_t = \{r_{1,t} \dots r_{50,t}\}$. The first step is to modify Equations 1 and 2 so that they fit with these purposes. The first choice that is common is to assume that we do not have access to the control signal u ; we therefore drop it from the state transition equation. The second choice is to make it explicit that what we observe is not sensory feedback (per se), but the firing rates of our neural population. To make this more explicit, I have changed the notation from y to r . Furthermore, because there will be baseline differences in the firing rates of each neuron, we add an offset (i.e., a one) to the kinematic vector x_t (see above). The state transition and sensory observation equations can therefore be written as follows:

$$x_{t+1} = Ax_t + w \quad (9)$$

$$r_{t+1} = Hx_{t+1} + q \quad (10)$$

Together we have four parameters, $\theta = \{A, H, W, Q\}$, that specify how the system itself (in the present case, the cursor) and the sensory observation evolves over time. Determining the values of these parameters is the first step to decode kinematics from firing rate. If we assume that their values are stable over time, we have closed form solutions and therefore they can be learned very quickly from training data using linear regression. We do not need to do this at every time step t but can do so using matrix algebra. The matrix X corresponds to the four kinematic variables (and the offset) at each time point and thus has the dimensions $5 \times T$. The matrix R corresponds to the firing rates of all fifty M1 neurons over time and thus has the dimensions $50 \times T$.

Let's first start with the **sensory observation equation**. As discussed in lecture, this is equivalent to a linear encoding model mapping X to R . Using linear regression to derive the variables H and Q , this corresponds to:

$$H = RX^T(XX^T)^{-1} \quad (11)$$

$$Q = \frac{1}{T}(R - HX)(R - HX)^T \quad (12)$$

Let's now turn to the **state transition equation**. In most monkey studies on BC/MI, the decoder is fit using the actual kinematics of the arm, X . For simplicity, this approach assumes that the desired cursor trajectory matches the arm dynamics. We can therefore fit our state transition variables using the actual recorded kinematics of the arm. Because the state transition matrix A corresponds to the *transitions* between states from one time point to the next, we can derive it from versions of the kinematic matrix X that have been offset in time. To make the notation a bit easier, we will make two new matrices: $X1 = X_{1:T-1}$ and $X2 = X_{2:T}$. Put another way, $X1$ is the kinematics from the first time point ($t = 1$) to the second-to-last time point ($t = T - 1$) and $X2$ is the kinematics from the second time point ($t = 2$) to the last time point ($t = T$). Using the temporally offset kinematics, our state transition matrix can be obtained via linear regression as follows:

$$A = X_2 X_1^T (X_1 X_1^T)^{-1} \quad (13)$$

$$W = \frac{1}{T-1} (X_2 - AX_1)(X_2 - AX_1)^T \quad (14)$$

We assume that the system dynamics are stable and therefore the above parameters are derived from the training set and fixed during testing. All other parameters vary from timepoint to timepoint and are thus determined uniquely with the test set. This is particularly true of the **state estimation equation** (Equations 5-8), which formalizes the relationship between *a priori* and *a posteriori* estimates and the evolving uncertainty in these estimates. To determine these values and decode the *a posteriori* state of the cursor, we iterate over all time points in the test set; for example, by using a *for loop*. The iteration process goes from the transformation of x_t^* to x_{t+1}^* (Equations 3 & 9) to the formation of the *a posteriori* estimate $\hat{x}_{t+1}$. This is your decoded value for that time point.

One crucial point that I need to emphasize is that all posterior variables at the end of the state estimation process become the prior variables of the next time step. Specifically, $\hat{x}_{t+1}$ becomes x_t^* and P_{t+1} becomes P_t . Of course, for the first time step (i.e., $t + 1 = 1$), there is no previous time step. We therefore initialize these variables at the beginning of the decoding: (1) x_0^* can be set either to a state of the cursor just prior to the training set or a reasonable location where the cursor could be; this choice doesn't usually matter for decoding performance, since the Kalman filter will quickly correct the error. (2) The covariance matrix P_0 determines the uncertainty of the *a posteriori* estimate, but since it is time zero we do not have one yet. Therefore, it is common to initialize the variances with high values (e.g., > 1000) to mimic a uniform prior. The specific high value does not matter so much, but it is important that it is high. If you initialize with a low value (i.e., high certainty) the decoder could perform worse (at least initially).

Two important points to note:

First, though Equation 9 specifies the actual state of the limb, our decoder only has access to this during training (i.e., when estimating the parameters). During the actual decoding itself (i.e., testing), the decoder obviously doesn't know the actual state of the limb since this is what we are trying to figure out! Instead we utilize the forward model to make predictions about the states and state estimation to come to a final decoded intended state, $\hat{x}_{t+1}$. Therefore, when programming the testing portion of your decoder, rewrite Eq 9 to be: $x_{t+1}^* = Ax_t^* + w$

Second, a brief word about the noise parameters, w and q . Equations 9 and 10 include noise terms for the state transition and the observation. The noise terms are included for completeness, as we assume that all measurements made by the nervous system (be they kinematics, firing rates, etc) are noisy. However, when you formulate the equations in the decoding procedure you drop them (e.g. Equation 9 becomes: $x_{t+1}^* = Ax_t^*$). This is because we are making measurements after the noise has already corrupted the signal, and so the noiseless values are unknown to us. The different sources of noise (in the form of covariance matrices W and Q ; see Eqns 12 and 14) do become important when calculating the Kalman gain (Eqns 6-8). These are the only place in the decoding algorithm where you make use of the covariances, and so make sure to include them there.

Homework assignment

The purpose of this assignment is to give you first-hand experience using a Kalman filter to decode neural signals during movement. You will do so with real brain and body recordings from an actual study (Chowdhury et al 2020, eLife). The equations and their description above should be sufficient to recreate the decoder (though ask me or Huseyin if you have questions!).

I have included a .mat file (KalmanDecoding.mat) with reaching kinematics, $\{1, x_t, y_t, \dot{x}_t, \dot{y}_t\}$, and simultaneously recorded the firing rates of 113 neurons from somatosensory Area 2. I apologize for those who prefer to use Python! There are functions you can use that import .mat files into Python, but please let us know if you need help on this.

All relevant matrices are divided into training sets and test sets. The kinematic matrices correspond to the variables X_{train} and X_{test} and the firing rate matrices correspond to the variables R_{train} and R_{test} . Each column of these matrices is a single discrete time step of 50 ms. I also include the two initializations described above, P_0 and x_0 , that you can use on the initial time step to start your decoder off.

Below I include several questions aimed at helping you get an intuition on the decoding. For each question, include a plot of the results along with your responses to specific sub questions. The lecture slides include some formats of plots that I recommend emulating (for example, see slides 10 and 18 in the lecture pdf document).

Question 1: Using the above equations, decode the actual kinematics (X_{test}) from the neural firing observations (R_{test}). First you should use the training variables to determine the system dynamics (i.e., use X_{train} and R_{train}). Then iterate over each time point of the test set, with the final 'decoded' kinematic variables for that time point corresponding to the *a posteriori* estimate. Make sure to save these values, so that you can assess how your decoded kinematics matches up with reality (i.e., X_{test}). Use R^2 to assess the goodness of fit of your decoder. What was your R^2 for each kinematic variable? Plot the decoding results for 100 time-steps in the format of slide 10 (lecture pdf).

Question 2: The values of the Kalman gain evolve over time but eventually stabilizes when the uncertainty of the estimates becomes low. Approximately how many time steps does it take for each to stabilize? Plot the Kalman gain for the first 100 or so time steps (as in slide 18 of the lecture pdf). You'll notice that for some kinematic variables the Kalman gain is positive while others it is negative. Please provide a functional explanation of this fact (i.e., Why would this be the case?).

Note: There is one Kalman gain for every neuron-kinematics pair (i.e., there are 5×113 of them!). To get a better grasp on its evolution, take the mean Kalman gain across neurons. This will give you one overall Kalman gain per four kinematic variable (ignore the baseline), which you can plot.

Question 3: Let's explore how the initialization choices affect decoding performance. Initialize your decoder using four combinations of x_0 and P_0 . Set the same value for each kinematic variable.

(1) $x_0 = 0$, $P_0 = 1000$; note that these are the initial values I provided you in the mat file. (2) $x_0 = -100$, $P_0 = 1000$; (3) $x_0 = 0$, $P_0 = 0.01$; and finally, (4) $x_0 = -100$, $P_0 = 0.01$.

What affect does each initialization pair have on immediate performance? Are some combinations better/worse than others? For each of the four combinations, plot the decoding results in the same format as in Question 1.

Question 4. In real neural recordings, the level of observation noise can differ significantly from training to testing conditions due to factors like electrode impedance changes, neural adaptation, or physiological state variations. To simulate a situation like this, scale the Q matrix of our decoder (from Question 1 and 2) by the following values: 0.01, 0.1, 1, 10, and 100. Answer the following subquestions based on the effect of different levels of scaling:

- A. How does each level of noise scaling change the *goodness of fit* (i.e., R^2) for the decoding of each kinematic variable (x position, y position, $\dot{x}$ velocity, $\dot{y}$ velocity)? Plot the results for at least one kinematic variable.
- B. How does each level of noise scaling change the evolution Kalman gain of each kinematic variable (x position, y position, $\dot{x}$ velocity, $\dot{y}$ velocity)? As in Question 2, plot the results for at least one kinematic variable.
- C. What is the relationship between observation noise, Kalman gain, and decoding performance? Why?
- D. Do the position or velocity variables show greater sensitivity to noise changes? Why might this be the case?